Relatório do Trabalho Prático I

Integração de Sistemas de Informação · Licenciatura em Engenharia de Sistemas Informáticos · Escola Superior de Tecnologia do IPCA · 2026/27

Autor: [Nome completo] · Nº [número]

Repositório Git: https://github.com/13andre/TP_ISI_2026

Integração e análise das seleções do Mundial de 2026

Conteúdo

Relatório do Trabalho Prático I 1

Integração e análise das seleções do Mundial de 2026 1

Resumo 3

1. Enquadramento 3

2. Problema 3

3. Estratégia utilizada 4

3.1 Arquitetura em três fases 4

3.2 Tecnologias 4

3.3 Modelo de dados 5

3.4 Operadores utilizados 5

3.5 Decisões de desenho 6

4. Transformações 6

4.1 fase1_csv 6

4.2 fase1_json 7

4.3 fase2_classificacao 7

4.4 fase2_alertas 8

4.5 fase3_exportacao 8

4.6 Validação cruzada entre as duas fontes 9

5. Jobs 9

6. Vídeo com demonstração 10

7. Conclusão e trabalhos futuros 10

7.1 Conclusão 10

7.2 Limitações 10

8. Referências 11

Resumo

Este trabalho implementa um processo ETL (Extract, Transform, Load) com o Pentaho Data Integration (Kettle) sobre os dados do Campeonato do Mundo de 2026. Dados em dois formatos (CSV e JSON) são carregados para uma base de dados SQL Server, cruzados e validados entre si. A partir deles calcula-se a classificação da fase de grupos, identificam-se alertas de resultados (goleadas) e exportam-se os resultados em CSV, JSON e XML. Um Job orquestra todo o processo, com tratamento de erro e registo em log.

1. Enquadramento

Item	Descrição
Unidade curricular	Integração de Sistemas de Informação (ISI)
Curso	Licenciatura em Engenharia de Sistemas Informáticos, Escola Superior de Tecnologia do IPCA
Ano letivo	2026/27
Equipa	[Nome(s) e número(s); indicar se o trabalho é individual ou em grupo]
Plataforma ETL	Pentaho Data Integration (Kettle) Community 10.2
Base de dados	SQL Server Express (base mundial2026)
Repositório	https://github.com/13andre/TP_ISI_2026

2. Problema

Os dados de uma competição desportiva chegam em origens e formatos diferentes: listas de seleções, calendários, resultados, eventos de jogo. Para os analisar é preciso integrá-los, validá-los e transformá-los numa forma que responda a perguntas concretas. O objetivo deste trabalho é construir um processo ETL que, para o Mundial de 2026, responda a três questões:

- Em que grupo joga cada seleção, a que confederação pertence e como ficou classificada no final da fase de grupos?
- As duas fontes de dados utilizadas (CSV e JSON) concordam entre si?
- Que jogos merecem ser assinalados como alertas de resultado (goleadas)?

Âmbito. Seleções e jogos: grupos, classificações, confederações e alertas de resultados. Os dados provêm de um dataset público (licença CC0) com 48 seleções, 104 jogos, 16 estádios, 7 fases do torneio, 601 eventos de jogo e um ficheiro JSON com o detalhe dos jogos.

Limites. O torneio já terminou, por isso os dados são estáticos e os alertas são calculados sobre os resultados finais, e não em tempo real. A classificação cobre apenas a fase de grupos (72 jogos), porque nas eliminatórias os empates são decididos por penáltis e a contagem de pontos deixa de se aplicar.

3. Estratégia utilizada

3.1 Arquitetura em três fases

Fase	Objetivo	Processos	Resultado
Fase 1 – Extract	Ler CSV e JSON e guardar em staging	fase1_csv, fase1_json	Seis tabelas stg_
Fase 2 – Transform	Validar, cruzar, calcular classificação e alertas	fase2_classificacao, fase2_alertas	Tabela classificacao_grupo e alertas
Fase 3 – Load	Persistir e exportar	fase3_exportacao	Ficheiros CSV, JSON e XML
Orquestração	Executar tudo por ordem, com erro e log	main_job	Log de execução

Os dados lidos na Fase 1 são guardados em tabelas de staging na base de dados, em vez de passarem diretamente à Fase 2. Isto torna cada fase independente (a Fase 2 começa sempre dos mesmos dados guardados), permite repetir as transformações sem voltar a ler os ficheiros e possibilita validações em SQL.

3.2 Tecnologias

- Pentaho Data Integration (Kettle) Community 10.2: Spoon (interface), Kitchen (execução de jobs).
- SQL Server Express e SQL Server Management Studio, com o driver JDBC da Microsoft (mssql-jdbc).
- Git e GitHub para o repositório.

3.3 Modelo de dados

Tabela	Origem	Linhas	Descrição
stg_teams	teams.csv	48	Seleções, grupo e confederação
stg_matches	matches.csv	104	Jogos e resultados
stg_venues	venues.csv	16	Estádios
stg_tornement_stages	tournament_stages.csv	7	Fases do torneio
stg_match_events	match_events.csv	601	Eventos de jogo
stg_match_details	real_match_details.json	104	Resultados do ficheiro JSON
classificacao_grupo	Calculada	48	Classificação final por grupo

3.4 Operadores utilizados

Operador	Onde	Função
CSV file input	fase1_csv	Leitura dos cinco CSV
JSON input	fase1_json	Extração de campos com JSONPath
Table input / Table output	Todas	Leitura e escrita na base de dados (Truncate e insert)
Select values	fase2_classificacao	Renomear campos (perspetiva casa e visitante)
Append streams	fase2_classificacao	Juntar os dois ramos (144 linhas)
Formula	classificacao e alertas	Pontos, vitórias, empates, derrotas, diferença de golos, texto do alerta
Stream lookup	fase2_classificacao	Acrescentar nome, grupo e confederação
Sort rows / Group by	fase2_classificacao	Ordenar e agregar por seleção
Add value fields changing sequence	fase2_classificacao	Posição dentro de cada grupo
Regex evaluation	fase2_alertas	Validar o formato da hora
Filter rows	fase2_alertas	Horas válidas e goleadas
Database lookup	fase2_alertas	Nome das seleções a partir de stg_teams
Text file output / JSON	fase2_alertas e fase3	Exportação

Operador	Onde	Função
output / XML output		
Write to log	fase2_alertas e Job	Registo de alertas e da conclusão
Start, Transformation, Write to log, Abort job	main_job	Orquestração e tratamento de erro

3.5 Decisões de desenho

- **Idempotência.** Todos os Table output usam Truncate table, para o processo poder correr várias vezes sem duplicar linhas.
- **Copy em vez de Distribute.** Quando um passo alimenta vários destinos, o movimento de dados é Copy, para cada destino receber todas as linhas.
- **Caminhos relativos.** Os ficheiros são referenciados com a variável ${Internal.Entry.Current.Directory}, para o projeto correr em qualquer computador que mantenha a estrutura de pastas.
- **Apenas fase de grupos.** A classificação usa os 72 jogos com stage_id = 1.
- **Validação cruzada.** Como existem duas fontes para o mesmo conjunto de jogos, os resultados e os nomes das seleções são comparados.

4. Transformações

4.1 fase1_csv

[INSERIR PRINT: diagrama da transformação fase1_csv]

A transformação tem cinco fluxos independentes e paralelos. Cada um lê um CSV com um passo CSV file input (delimitador vírgula, codificação UTF-8, cabeçalho presente) e grava-o com um Table output numa tabela de staging, com Truncate table. Os tipos dos campos foram definidos com o botão Get Fields e revistos: identificadores como Integer (comprimento 15, precisão 0) e textos como String.

Leitura	Tabela	Linhas
teams	stg_teams	48
matches	stg_matches	104
venues	stg_venues	16
tournament_stages	stg_tornement_stages	7
match_events	stg_match_events	601

[INSERIR PRINT: Step Metrics da fase1_csv]

4.2 fase1_json

[INSERIR PRINT: diagrama da transformação fase1_json]

O ficheiro real_match_details.json é uma lista de 104 objetos, um por jogo. O passo JSON input lê o ficheiro com caminhos do tipo $[*].campo e extrai cinco campos: match_id, home_team, away_team, home_score e away_score. O resultado é gravado em stg_match_details (104 linhas). Cada jogo contém ainda listas aninhadas com os golos (home_goals e away_goals); como têm um número de elementos diferente por jogo, não podem ser lidas no mesmo passo que os campos simples, e ficam como trabalho futuro.

4.3 fase2_classificacao

[INSERIR PRINT: diagrama da transformação fase2_classificacao]

Esta é a transformação central. A ideia é converter cada jogo em duas linhas, uma por seleção, para depois somar por seleção.

1. **Ler jogos da fase de grupos** (Table input): seleciona de stg_matches os jogos com stage_id = 1 (72 jogos).
2. **home_team e away_team** (dois Select values em paralelo, com movimento Copy): o primeiro renomeia home_team_id para team_id, home_score para goals_scored e away_score para goals_conceded; o segundo faz o inverso para a equipa visitante. Os dois ramos têm os mesmos campos pela mesma ordem.
3. **join_teams** (Append streams): empilha os dois ramos, produzindo 144 linhas (cada jogo aparece duas vezes, uma por seleção).
4. **points** (Formula): calcula, por linha, os pontos com IF([goals_scored]>[goals_conceded];3;IF([goals_scored]=[goals_conceded];1;0)), as colunas wins, draws e losses (1 ou 0) e goal_difference.
5. **Ler selecoes** e **Stream lookup**: um Table input lê team_id, team_name, group_letter e confederation de stg_teams, e o Stream lookup acrescenta esses campos a cada linha pelo team_id (192 linhas lidas, 144 escritas).
6. **Sort rows**: ordena por team_id, necessário porque o Group by só agrega linhas consecutivas.
7. **Group by**: agrupa por team_id, team_name, group_letter e confederation e calcula played (contagem), wins, draws, losses, goals_for, goals_against, goal_difference e points (somas). Saem 48 linhas.
8. **ranking_stands** (Sort rows): ordena por group_letter ascendente e depois por points, goal_difference e goals_for descendentes.
9. **Add value fields changing sequence**: cria a coluna position, que recomeça em 1 sempre que o grupo muda.
10. **Table output**: grava em classificacao_grupo (48 linhas, 12 grupos de 4, played = 3 em todas).

[INSERIR PRINT: Step Metrics da fase2_classificacao]

[INSERIR PRINT: resultado da classificacao_grupo no SQL Server Management Studio]

4.4 fase2_alertas

[INSERIR PRINT: diagrama da transformação fase2_alertas]

Esta transformação identifica os jogos com goleada e usa uma expressão regular para validar os dados.

1. **Ler jogos** (Table input): seleciona de stg_matches o identificador, a data (convertida para texto), a hora, os identificadores das seleções e os golos (104 linhas).
2. **validar hora** (Regex evaluation): aplica ao campo kickoff_time_utc a expressão regular abaixo, que aceita horas de 00:00 a 23:59, e guarda o resultado em hora_valida.

```
^([01]\d|2[0-3]):[0-5]\d$
```

3. **so horas validas** (Filter rows): deixa passar as linhas com hora_valida = Y. Todas as 104 linhas passaram.
4. **nome casa** e **nome fora** (Database lookup): obtêm de stg_teams o nome das seleções (home_team e away_team).
5. **calcular diferenca** (Formula): goal_diff = ABS([home_score]-[away_score]).
6. **so goleadas** (Filter rows): goal_diff >= 4.
7. **montar alerta** (Formula): compõe o texto do alerta.
8. Os alertas são gravados em export/alertas.csv (Text file output, UTF-8) e escritos no log (Write to log).

A regra de negócio, diferença de golos igual ou superior a 4, devolve quatro alertas:

Data	Alerta
2026-06-14	Germany 7-1 Curaçao
2026-06-18	Canada 6-0 Qatar
2026-06-23	Portugal 5-0 Uzbekistan
2026-06-26	Senegal 5-0 Iraq

[INSERIR PRINT: pré-visualização do campo alerta e conteúdo de alertas.csv]

4.5 fase3_exportacao

[INSERIR PRINT: diagrama da transformação fase3_exportacao]

Um Table input lê a classificacao_grupo, ordenada por grupo e posição, e envia as 48 linhas (movimento Copy) para três passos de saída em paralelo: export CSV (Text file output), export JSON (JSON output, num único bloco) e export XML (XML output, com elemento de linha equipa). Os ficheiros são escritos em export/classificacao.csv, export/classificacao.json e export/classificacao.xml, com codificação UTF-8.

[INSERIR PRINT: Step Metrics da fase3_exportacao e pasta export]

4.6 Validação cruzada entre as duas fontes

As duas fontes descrevem os mesmos 104 jogos, e foram comparadas com consultas SQL (sql/03_consultas_validacao.sql):

Verificação	Resultado
Jogos com resultado diferente entre CSV e JSON	0 linhas
Equipas da casa do JSON sem correspondência em stg_teams	0 linhas
Equipas visitantes do JSON sem correspondência em stg_teams	0 linhas
Jogos que existem só numa das fontes	0 linhas

As fontes coincidem nos 104 jogos, nos resultados e nos nomes das seleções.

[INSERIR PRINT: consultas de validação no SQL Server Management Studio]

5. Jobs

[INSERIR PRINT: diagrama do main_job]

O Job main_job liga as cinco transformações por esta ordem: Start → fase1 → fase1_json → fase 2 → fase2_alertas → fase 3 → Write to log. Cada entrada de transformação referencia o ficheiro com ${Internal.Entry.Current.Directory}, para ser independente do computador.

- Os hops entre transformações são condicionais (seguir apenas se o resultado for verdadeiro).
- Cada transformação tem ainda um hop de falha para um Abort job, que termina o processo com erro se algum passo falhar.
- O Write to log final regista a mensagem «Processo ETL concluído: classificação gerada e exportada».

O Job pode ser executado no Spoon ou pela linha de comandos com o Kitchen, que guarda a execução em logs/execucao.log:

```
Kitchen.bat /file:<caminho>\workflows\main_job.kjb /level:Basic
/logfile:<caminho>\logs\execucao.log
```

[INSERIR PRINT: Job metrics com todas as entradas em Sucesso]

[INSERIR PRINT: excerto do log de execução]

6. Vídeo com demonstração

[INSERIR QR CODE E LINK DO VÍDEO]

O vídeo mostra os dados de entrada em raw/, a execução do main_job no Spoon até ao fim, as tabelas no SQL Server Management Studio e os ficheiros gerados em export/.

7. Conclusão e trabalhos futuros

7.1 Conclusão

Foi construído um processo ETL completo, que lê dados de duas fontes e formatos, valida-os por cruzamento, calcula a classificação por grupo (com confederação), identifica alertas de goleadas e exporta os resultados em três formatos, tudo orquestrado por um Job com tratamento de erro e log. A comparação entre as fontes não encontrou nenhuma discrepância, o que dá confiança nos resultados.

Critério do enunciado	Como foi cumprido
Formatos de entrada	CSV (cinco ficheiros) e JSON
Fontes de dados	Ficheiros locais e base de dados relacional (Table input e lookups)
Expressões regulares	Validação do formato da hora em fase2_alertas
Serialização JSON e XML	Leitura de JSON; exportação em JSON e XML
Joins, lookups e agrupamentos	Stream lookup, Database lookup, Append streams, Group by
Operações sobre valores	Renomear, calcular, filtrar, ordenar, concatenar
Geração de logs	Write to log e ficheiro logs/execucao.log
Orquestração	main_job com tratamento de erro
Operações sobre base de dados	Criação de tabelas, Truncate e insert, consultas de validação

7.2 Limitações

- A classificação só considera a fase de grupos e os critérios pontos, diferença de golos e golos marcados; não aplica os desempates adicionais da FIFA (confronto direto, fair play).
- A confederação é um atributo da classificação, mas não foi feita uma agregação por confederação.
- Os dados são estáticos, por isso os alertas são calculados em lote e não em tempo real.

8. Referências

1. Ferreira, L. e Ribeiro, Ó. (2026). Integração de Sistemas de Informação, Trabalho Prático I (enunciado). Escola Superior de Tecnologia do IPCA.
2. mominullptr. FIFA World Cup 2026 Dataset (licença CC0). GitHub. https://github.com/mominullptr/FIFA-World-Cup-2026-Dataset
3. Pentaho Data Integration (Kettle) Community 10.2, documentação oficial. [inserir URL e data de consulta]
4. Microsoft. JDBC Driver for SQL Server (mssql-jdbc). [inserir URL e data de consulta]
5. Microsoft. SQL Server Express e SQL Server Management Studio, documentação. [inserir URL e data de consulta]